

CSE 318 Assignment-02: Max-Cut Algorithms Report

Student ID: 2105019

May 2025

1 Introduction

This report presents the implementation and analysis of five Max-Cut algorithms for the CSE 318 Assignment-02. The algorithms were applied to 54 benchmark graphs from the `set1` directory, and their performance metrics (cut weights and iterations) are recorded in `2105019.csv`. The report includes a description of each algorithm, a comparison of their performance, and visualizations of the results for graphs G1 to G10.

2 Algorithm Descriptions

2.1 Randomized Heuristic

The Randomized Heuristic randomly assigns each vertex to one of two partitions (X or Y) with equal probability (0.5). This process is repeated 50 times, and the best cut weight is selected. It is fast and simple but often produces suboptimal solutions due to its lack of strategic assignment.

2.2 Greedy Heuristic

The Greedy Heuristic starts by selecting the edge with the maximum weight and assigns its endpoints to different partitions. It then iteratively assigns each remaining vertex to the partition that maximizes the cut weight contribution (sum of edge weights to the opposite partition). It is deterministic and efficient but may get stuck in local optima.

2.3 Semi-Greedy Heuristic

The Semi-Greedy Heuristic extends the Greedy approach by introducing randomness. It uses a Restricted Candidate List (RCL) based on a threshold $\mu = w_{\min} + \alpha(w_{\max} - w_{\min})$, where $\alpha = 0.5$, and randomly selects a vertex from the RCL to assign to a partition. This balances greediness and randomness, often yielding better results than the Greedy method.

2.4 Local Search

Local Search improves an initial solution (from the Greedy Heuristic) by iteratively moving a vertex to the other partition if it increases the cut weight. It stops when no such move exists, reaching a local optimum. It enhances the Greedy solution but is limited by its local search scope.

2.5 GRASP

GRASP (Greedy Randomized Adaptive Search Procedure) combines the Semi-Greedy Heuristic with Local Search. It runs for 50 iterations, each iteration constructing a solution using the Semi-Greedy approach ($\alpha = 0.5$) and improving it with Local Search. The best solution across iterations is returned. GRASP often achieves the highest cut weights by exploring diverse solutions.

3 Comparison of Algorithms

The performance of the algorithms was evaluated on 54 graphs (G1 to G54), with results recorded in `2105019.csv`. Key observations include:

- **Overall Performance:** GRASP consistently achieves the highest cut weights across most graphs, followed by Local Search, Semi-Greedy, Greedy, and Randomized. For example, on G1 ($|V| = 800$, $|E| = 19176$), GRASP achieves a cut weight of 11444, compared to 11322 for Local Search, 11102 for Semi-Greedy, 10956 for Greedy, and 9602 for Randomized.
- **Closeness to Known Best:** For graphs with known best solutions (e.g., G1: 12078), GRASP comes closest (11444, 94.8% of the known best), while Randomized performs the worst (9602, 79.5%).
- **Negative Weights:** Graphs G6 to G10 and others with negative weights (e.g., G7: -66 for Randomized) show poor performance for Randomized, as random assignments can lead to negative cut weights. Greedy and Semi-Greedy handle negative weights better by focusing on maximizing contributions.
- **Iterations:** Local Search requires 0 to 257 iterations (e.g., 0 for G48, 257 for G31), while GRASP uses a fixed 50 iterations per run (as per the implementation).
- **Sparse vs. Dense Graphs:** On sparse graphs (e.g., G11, $|E| = 1600$), all algorithms perform closer to the known best (e.g., G11: GRASP at 480 vs. 627). On denser graphs (e.g., G22, $|E| = 19990$), the gap widens (GRASP at 12994 vs. 14123).

3.1 Summary Table

The table below summarizes the performance for selected graphs (G1, G22, G43):

Graph	Randomized	Greedy	Semi-Greedy	Local Search	GRASP	Known Best
G1	9602	10956	11102	11322	11444	12078
G22	9988	12346	12666	12897	12994	14123
G43	5006	6188	6340	6485	6510	7027

Table 1: Cut weights for selected graphs.

4 Visualizations

To visualize the performance, two plots were generated using Python and Matplotlib for graphs G1 to G10:

4.1 Plot 1: Cut Weights Across Graphs

This bar plot compares the cut weights of all algorithms for graphs G1 to G10

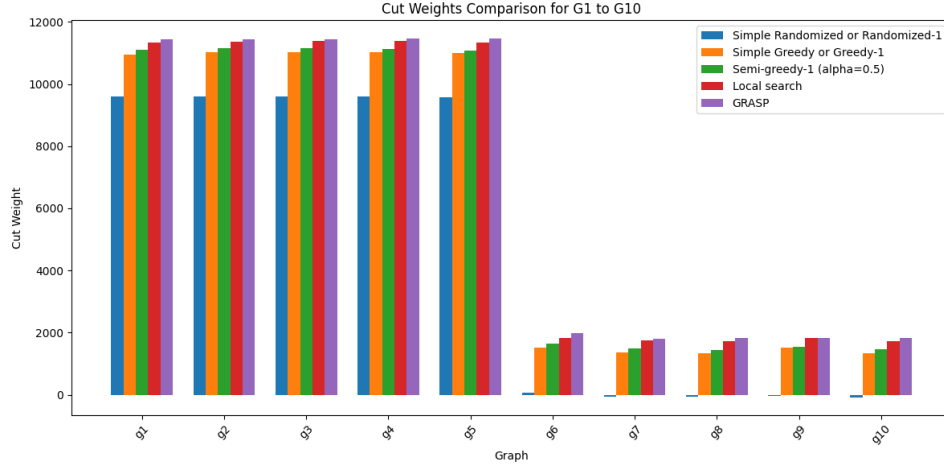


Figure 1: Comparison of cut weights for G1 to G10.

5 Conclusion

GRASP consistently outperforms other algorithms, achieving the highest cut weights due to its combination of randomization and local optimization. Local Search improves Greedy solutions effectively, while Semi-Greedy offers a balance between randomness and greediness. Randomized performs the worst, especially on graphs with negative weights. Future improvements could include dynamic α for Semi-Greedy or multiple GRASP runs for better averages.